

# build\_image

## build\_image.sh — user guide

**Revision:** 1 **Last modified:** 2026-06-11T00:00:00Z

Companion documentation (§11.4.18) for tests/emulator/ab\_virt/build\_image.sh.

### Overview

build\_image.sh builds a bootable **aarch64 Linux guest image** for the dev-host RK3588 / Orange Pi 5 Max A/B emulator. The produced kernel + rootfs boot under QEMU `-machine virt + HVF` on an Apple-Silicon host and are the foundation the A/B layers (U-Boot bootcount + RAUC dm-verity) build on.

The script's current target is phase **PWU-AB-0**: a base **Buildroot aarch64** rootfs + kernel with dropbear SSH and a known root credential, plus the A/B toolchain pieces (U-Boot qemu\_arm64, RAUC, dosfstools / e2fsprogs / util-linux) compiled in from the qemu\_aarch64\_virt\_defconfig base configuration.

Because the build host is macOS while Buildroot needs a Linux build host, the build runs **inside a podman aarch64 Linux container** (native arm64, no emulation) using Buildroot's **internal toolchain** (not the Bootlin external toolchain, which is invalid for this defconfig).

### Prerequisites

- podman on PATH (the only hard host dependency; the script aborts with exit 3 if absent).
- A podman machine with roughly **10–20 GB** of free disk for the toolchain + kernel + rootfs build.
- Network access (Buildroot downloads the toolchain + package sources).

### Usage examples

```
# Build the base image (long: toolchain + kernel + rootfs).
tests/emulator/ab_virt/build_image.sh
```

```
# Wipe out/ + the download-cache volume + a stale build container,
  then build.
tests/emulator/ab_virt/build_image.sh --clean

# Override the Buildroot version and the root password.
BR2_VERSION=2024.02.10 HELIX_AB_ROOT_PW=helixota \
  tests/emulator/ab_virt/build_image.sh
```

## Inputs

| Input            | Source        | Default                    |
|------------------|---------------|----------------------------|
| --clean          | CLI arg 1     | (none) — incremental build |
| BR2_VERSION      | env           | 2024.02.10                 |
| HELIX_AB_ROOT_PW | env → ROOT_PW | helixota                   |

The script also uses two fixed podman object names: a named download-cache volume `helix_ab_dl` and a named build container `helix_ab_build`.

## Outputs

All outputs land under `tests/emulator/ab_virt/out/` and are gitignored (§11.4.30; the script itself is the §11.4.77 regeneration mechanism):

| Output                              | Meaning                                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>out/images/Image</code>       | aarch64 kernel image                                                                              |
| <code>out/images/rootfs.ext2</code> | Buildroot rootfs                                                                                  |
| <code>out/images/u-boot.bin</code>  | U-Boot qemu_arm64 bootloader<br>(additive — extracted if present)                                 |
| <code>out/build.log</code>          | full build transcript (stdout+stderr of<br>the container run + podman cp)                         |
| <code>out/.ok</code>                | success stamp — written <b>only</b> when the<br>real kernel + rootfs are present and<br>non-empty |

## Side-effects

- Creates the named podman volume `helix_ab_dl` (download cache; persists across runs so re-builds do not re-download).
- Creates and (on success/failure paths) removes the named build container `helix_ab_build`.
- --clean additionally `rm -rf out/` and force-removes the volume + container.
- Writes the two image artifacts host-side via `podman cp` (no bind-mount; works on the external `/Volumes/T7` volume that is not shared into the podman machine VM).

## Internal behaviour

1. `set -u + set -o pipefail`; resolve `SCRIPT_DIR`, `OUT`, the Buildroot version, root password, and the volume/container names.
2. Abort (exit 3) if `podman` is not found.
3. On `--clean`, wipe `out/`, the download volume, and a stale container.
4. `mkdir -p out/images`; remove any prior `out/.ok` so a stale stamp can never survive a failed run.
5. `podman volume create helix_ab_dl` (idempotent) and force-remove any leftover build container.
6. `podman run --name helix_ab_build --arch arm64` against `debian:bookworm-slim` with the download volume mounted at `/dl`:
  - `apt-get` installs the Buildroot host toolchain build deps plus **libssl-dev** (U-Boot host tools `mkimage/aisimage` need `openssl/evp.h`) and **bison/flex** (U-Boot `Kconfig` / `dtc` parser generators).
  - Buildroot refuses to build as root, so a dedicated `br` user owns `/work` and `/dl`.
  - As `br`: download + extract Buildroot, make `qemu_aarch64_virt_defconfig`, append the A/B config fragment to `.config` (root password, dropbear + client, util-linux, `e2fsprogs` + `resize2fs`, `dosfstools` + `mkfs.fat`, RAUC, U-Boot `qemu_arm64` with DTC + `.bin` format), make `olddefconfig`, then make `-j$(nproc)`.
  - The whole transcript is redirected to `out/build.log`.
7. If the container run returns non-zero, log the last 25 lines of `build.log`, remove the container, and exit 1.
8. Extract Image and `rootfs.ext2` via `podman cp`; extract `u-boot.bin` if present (the `.ok` gate keys on kernel + `rootfs` only, so a missing U-Boot stays visible rather than faked). Remove the container.
9. **Anti-bluff gate (§11.4.6)**: stamp `out/.ok` and exit 0 **only** when both `out/images/Image` and `out/images/rootfs.ext2` exist and are non-empty. Otherwise log “images missing — NOT stamping `.ok`” and exit 1.

## Edge cases

- **podman absent** → exit 3 before any work.
- **Build fails inside the container** → exit 1 with a `build.log` tail; `.ok` is never written.
- **Build finishes but the image artifacts are missing/empty** → exit 1, `.ok` not stamped (the explicit anti-bluff guard — a phantom build is never claimed as success).
- **u-boot.bin not produced** → the `podman cp` for it is tolerated (`|| true`); the build can still succeed at the kernel+`rootfs` level, and the missing bootloader is observable by its absence in `out/images/`.

## Related scripts

- `tests/emulator/ab_virt/boot_smoke.sh` — boots the produced Image + `rootfs.ext2` under QEMU+HVF and asserts a live interactive userspace. See `docs/scripts/boot_smoke.md`.

- `tests/emulator/ab_virt/assemble_ab_disk.sh` — consumes `Image` + `rootfs.ext2` + `u-boot.bin` to assemble the 2-slot GPT disk. See `docs/scripts/assemble_ab_disk.md`.
- `docs/research/rk3588_emulator/REPORT.md` — the emulator design report.

## **Last verified date**

2026-06-11 — documented against the script as committed (read, not inferred).